BoolE: Exact Symbolic Reasoning via Boolean Equality Saturation

Jiaqi Yin*, Zhan Song*, Chen Chen, Qihao Hu, Cunxi Yu
University of Maryland, College Park
College Park, US
{jyin629,cunxiyu}@umd.edu

2025 62nd ACM/IEEE Design Automation Conference (DAC) | 979-8-3315-0304-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/DAC63849.2025.11132728

***Abstract*—Boolean symbolic reasoning for gate-level netlists is a critical step in verification, logic and datapath synthesis, and hardware security. Specifically, reasoning datapath and adder tree in bit-blasted Boolean networks is particularly crucial for verification and synthesis, and challenging. Conventional approaches either fail to accurately (exactly) identify the function blocks of the designs in gate-level netlist with structural hashing and symbolic propagation, or their reasoning performance is highly sensitive to structure modifications caused by technology mapping or logic optimization. This paper introduces BoolE, an exact symbolic reasoning framework for Boolean netlists using equality saturation. BoolE optimizes scalability and performance by integrating domain-specific Boolean ruleset for term rewriting. We incorporate a novel extraction algorithm into BoolE to enhance its structural insight and computational efficiency, which adeptly identifies and captures multi-input, multi-output high-level structures (e.g., full adder) in the reconstructed e-graph.**

Our experiments show that BoolE surpasses state-of-the-art symbolic reasoning baselines, including the conventional functional approach (ABC) and machine learning-based method (Gamora). Specifically, we evaluated its performance on various multiplier architecture with different configurations. Our results show that BoolE identifies $3.53\times$ and $3.01\times$ more exact full adders than ABC in carry-save array and Booth-encoded multipliers, respectively. Additionally, we integrated BoolE into multiplier formal verification tasks, where it significantly accelerates the performance of traditional formal verification tools using computer algebra, demonstrated over four orders of magnitude runtime improvements.

I. Introduction

Boolean symbolic reasoning, which extracts word-level abstractions from gate-level netlists, plays a critical role in electronic design automation (EDA) such as logic synthesis, datapath optimization, and formal verification [8], [20], [21]. Additionally, the globalization of VLSI design and manufacturing processes has amplified the need for detecting malicious logic, such as hardware Trojans, to ensure hardware security [22], [26]. However, conventional approaches, such as those based on cut enumeration [3], [25], [31], structural hashing [20], [21], [42], and machine learning (ML) techniques [1], [12], [35], [37], face several critical limitations when addressing complex reasoning tasks, particularly in *bit-blasted* non-linear arithmetic netlists:

(1) *Static structural limitation* – Structural approaches, such as those based on cut enumeration and local structural hashing [3], [25], [31], often use circuit topology for shape hashing to identify structurally similar wires and form word-level abstractions. Alternatively, they rely on reference libraries to map sub-circuits by matching local truth tables [5], [42]. In these cases, the original high-level structures of a given netlist—such as carry-chains and adder trees—are often fragmented or altered during heavy logic optimization and technology mapping, rendering them unrecognizable. **(2)** *Lack of completeness and exactness*: State-of-the-art (SOTA) Boolean methods for symbolic reasoning focus on detecting Negation-Permutation-Negation (NPN) classes of functional components, which represent groups of structurally similar but not exact functional blocks. While these abstractions are useful for technology mapping [14], they fail to provide exactness guarantees, which are critical for real-world applications like formal verification. Furthermore, ML-based approaches, such as Gamora [37], HOGA [12], and DeepGate [17], [29], lack completeness and correctness guarantees due to the inherently probabilistic nature of ML models. **(3)** *Scalability challenges of functional approaches*: Functional approaches using symbolic evaluation are solver-ready (e.g., SAT-based approaches [2], [11]), but incur significant computational overhead, especially when applied to *bit-blasted* non-linear arithmetic Boolean networks [15], [20], [40].

Given the limitations of conventional approaches, we present **BoolE**, an exact symbolic reasoning framework that utilizes Boolean *equality saturation*. BoolE is designed to enhance reasoning performance for complex Boolean netlists, which are technology-mapped or logic-optimized. It takes Boolean netlists as input and infers functional blocks through Boolean-level equality saturation and term rewriting. By leveraging the e-graph saturation theory [32], BoolE offers scalable solutions to reasoning high-level blocks in heavily optimized and technology mapped arithmetic multipliers. More importantly, BoolE offers formal exactness and correctness guarantees. The key contributions of BoolE are summarized as follows:

1) **Boolean reasoning through equality saturation:** To the best of our knowledge, BoolE is the first end-to-end framework that uses equality saturation to explore a vast space of equivalent Boolean expressions. BoolE methodologies are believed to have broader impacts beyond symbolic reasoning in Boolean domain.
2) **Novel exact extraction algorithm for multi-output structures:** BoolE introduces an innovative exact extraction algorithm alongside a domain-specific Boolean ruleset, capable of handling complex multi-input, multi-output high-level structures. This ensures precise and efficient logic reconstruction, with exactness and correctness guaranteed by equality saturation theory.
3) **Comprehensive evaluation on reasoning:** Extensive experimental results demonstrate that BoolE outperforms SOTA functional, structural, and ML-based approaches by significant margins in both exact and NPN reasoning.
4) **End-to-end integration to real-world application:** BoolE has been seamlessly integrated with SOTA synthesis and verification tools, such as ABC, as well as formal verification backends like RevSCA2.0 [21]. A case study on the formal verification of arithmetic multipliers demonstrates over **four orders of magnitude** runtime acceleration on RevSCA-2.0.

II. Background

A. Boolean Networks

Boolean networks are mathematical models that represent logical relationships between binary variables, where nodes signify Boolean

*: co-first authors. J. Yin and C. Yu contributed the idea. J. Yin implemented the framework. Z. Song conducted experimental and baseline benchmarking setups. All authors contributed to the writing.

variables (0 or 1) and edges denote dependencies defined by Boolean functions. These networks are fundamental in digital circuit design for modeling combinational logic, and be used in computational biology for gene regulatory networks. An And-Inverter Graph (AIG) is a specialized Boolean network extensively used in electronic design automation (EDA). It is a directed acyclic graph (DAG) composed solely of two-input AND gates and NOT gates (inverters). AIGs offer a compact representation of Boolean functions, facilitating efficient manipulation, optimization, and verification of complex digital circuits. All combinatorial Boolean networks can be converted to AIG with De Morgan's laws.

A cut of a Boolean network is a pair (r, S), where r is a node called the root, and S is a set of nodes called the leaves, such that: (1) Every path from a primary input to r passes through at least one leaf in S. (2) For each leaf $l \in S$, there is a path from a primary input to r that goes through l but no other leaf. The size of the cut is the number of leaves $|S|$. A cut is K-feasible if its size $|S| \leq K$. This work focuses on 3-feasible cuts. The cut enumeration algorithm is a fundamental technique in Boolean network analysis, specifically used in symbolic reasoning and technology mapping. By enumerating all possible K-feasible cuts, the cut enumeration algorithm systematically explores various sub-network configurations, enabling the detection of critical Boolean functions such as FAs and multiplexers. The exhaustive exploration enables the identification of optimal sub-functions, enhances logic optimization, and supports technology mapping by providing a comprehensive understanding of the circuit functional structure. Despite the effectiveness of the cut enumeration algorithm, it struggles to accommodate structural modifications such as technology mapping and logic optimization.

NPN classification is a method used in logic synthesis and technology mapping to group Boolean functions into equivalence classes [14]. Two Boolean functions are considered NPN equivalent if one can be transformed into the other through a combination of input negations, input permutations, and output negation. Each NPN class of a representative function comprises all functions that can be derived from these transformations.

B. Boolean Word-Level Block Identification

Boolean word-level block identification in digital design aims to detect elementary functional blocks (e.g., FA) within netlists, providing critical word-level abstractions that enable logical optimization, formal verification, malicious logic detection, and other verification tasks [7], [21], [22]. Conventional approaches include structural shape hashing and functional bitslice aggregation [18], [31], [42], which analyze block-level netlists from structural and functional perspectives. Structural shape hashing assigns a unique shape to each edge in a Boolean netlist to capture its structural properties. This shape is defined as a directed graph that represents the backward-reachable logic gates from the wire. Functional bitslice aggregation, on the other hand, relies on functional matching to group equivalent nodes and edges using the cut enumeration algorithm. The cut enumeration algorithm is integrated into synthesis tools like ABC. Besides, some works also focus on recover higher-level programming abstractions in hardware description language (HDL) code [27], [30]. Recently, Graph Neural Networks (GNNs) have emerged as a promising tool for Boolean block identification [1], [12], [13], [35], [37]. Gamora [37], for instance, leverages GNNs and GPU acceleration to efficiently infer high-level functional blocks from gate-level netlists.

FA is the fundamental functional block for adder trees, consisting of sum and carry signals. which are represented by XOR and MAJ gates with identical input signals. Both the cut enumeration algorithm in ABC and the Gamora framework identify FAs based on NPN equivalence classes. In this paper, we refer to the logically equivalent FA as **exact FA**, and NPN equivalent FA as **NPN FA**.

C. Equality Saturation

Equality saturation [23], [24], [32], [43] is a rewriting optimization technique powered by an e-graph (equivalence graph) data structure, which represents an equivalence relation over expressions. An **e-graph** $\mathcal{G}$ is formally defined as a tuple $\mathcal{G} = (V, \mathcal{R}, \lambda)$, where:

- V is a finite set of vertices, referred to as **E-nodes**. Each e-node within V represents a distinct expression or sub-expression.
- $\mathcal{R} \subseteq V \times V$ defines an equivalence relation on V, partitioning it into equivalence classes known as **E-classes**. These E-classes are the equivalence classes generated by $\mathcal{R}$. Nodes within the same e-class are considered equivalent under the relation $\mathcal{R}$.
- $\lambda : V \to \Sigma \times V^k$ is a labeling function that assigns each node to an operator and an ordered list of child nodes, where Σ represents the set of operators with k operands and V^k denotes an ordered tuple of k child nodes from V.

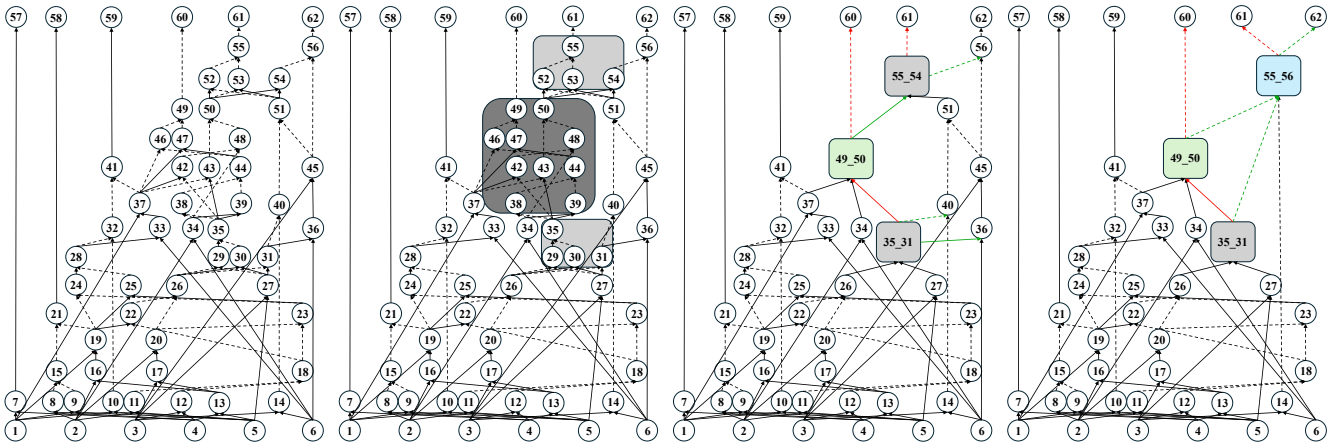
E-graphs form the foundation of **equality saturation** optimization technique, which applies rewrite rules iteratively until no new equivalences can be introduced. The process involves: (1) constructing an e-graph $\mathcal{G}_0$ from the initial expression s_0, (2) applying rewrite rules (l_i, r_i) to incorporate equivalences $[l_i]_\mathcal{R} \sim [r_i]_\mathcal{R}$, and (3) repeating until convergence, which indicates that no further changes can be made. E-graphs compactly store an exponential number of expressions in linear space by sharing sub-expressions, preserving all expressions and avoiding the phase-ordering problem [16]. Equality saturation has been applied in numerous areas within hardware design automation [4], [6], [9], [33], [34], [39]. For our implementation, we utilize the **egg** tool [36], an advanced e-graph framework, as the backend for BoolE e-graphs.

III. Motivating Example

In this section, we present a motivating example in Figure 1 to illustrate challenges in FA block identification. Figure 1(a) shows the AIG of a 3-bit carry-save-array (CSA) multiplier after ASAP 7nm technology mapping [38], with solid and dashed lines representing output signals and their negations, respectively. Prior to technology mapping, the 3-bit CSA multiplier netlist contains 3 FAs. However, after technology mapping, ABC identifies only one NPN FA block through cut enumerations, as shown in Figure 1(b). In this figure, the output of XOR and MAJ gates are highlighted in red and green, respectively. Figure 1(c) illustrates the adder tree of the 3-bit post-mapping CSA multiplier, where block `49_50` is identified as an NPN FA (marked in green), and blocks `55_54` and `35_31` are HAs (marked in gray).

While NPN classification effectively recognizes structurally similar components, it does not guarantee logical equivalence to true FAs. For tasks such as formal verification, which require precise logical equivalence to ensure the correctness of a design, identifying NPN-equivalent FAs is insufficient. Formal verification demands that the functional behavior of the circuit matches the specification exactly, without deviations introduced in NPN transformations. For example, RevSCA-2.0 [21] utilizes block identification of exact HAs and FAs to eliminate vanishing monomials by detecting and locally removing sources of redundant terms ahead of backward rewriting. However, this process requires exact logical equivalence to functional blocks.

Conventional Boolean reasoning methods, like cut enumeration, rely on static netlist structures to identify functional blocks such as FAs. However, processes like technology mapping and logic optimization often modify or eliminate these structures, rendering them



(a) AIG of 3-bit CSA multiplier w 7nm ASAP mapping.

(b) FA/HA block in the AIG.

(c) NPN FA block 49_50 and HA blocks {35_31,55_54} by cut enumeration.

(d) BoolE identifies an additional exact FA block 55_56 is extracted.

Fig. 1: Motivations for BoolE. The component blocks of exact FA, NPN FA, and HA are shown in blue, green, and grey blocks. The output signals of corresponding XOR and MAJ are marked as green and red correspondingly. (a) AIG of a 3-bit CSA multiplier generated by ABC and mapped using 7nm ASAP Technology Mapping. (b) FA and HA blocks in AIG detected by ABC. (c) Adder tree extracted from AIG. There is only one NPN FA. (d) The output netlist extracted after BoolE rewriting. An additional exact FA is reconstructed after BoolE rewriting.

unrecognizable to traditional tools. For the example in Figure 1, only one FA can be identified with cut enumeration. In contrast, equality saturation rewrites netlists into functionally equivalent forms, enabling exhaustive exploration of alternative structures. This flexibility opens opportunities for reconstructing functional blocks, thereby improving the robustness and effectiveness of symbolic reasoning tasks.

Finally, we present the netlist extracted from BoolE in Figure 1(d), where BoolE has reconstructed the netlist structure for FA identification. Apart from the NPN FA `49_50`, BoolE successfully identifies an additional exact FA, `55_56` (marked in blue), after term rewriting. For a large-scale 128-bit technology-mapped CSA multiplier, BoolE reconstructs 14,713 out of 16,128 NPN FAs within the theoretical upper bound, which is 3,771 more than what ABC can identify. Furthermore, 3,418 of the reconstructed FAs are exact FAs, representing a $4.42\times$ performance improvement compared to ABC.

IV. Approach

The overview of BoolE is illustrated in Figure 2. The framework accepts gate-level netlists as input, where each node represents a standard Boolean algebra operation (e.g., `AND`, `XOR`, `NOT`). Initially, BoolE parses the input AIG file, extracts necessary information, and converts the netlist into an e-graph. Subsequently, the BoolE rewriting engine applies rewriting rules to expand the e-graph, thereby identifying `MAJ` and `XOR` components. For e-graph extraction, we developed a DAG-based cost extraction algorithm to ensure the multi-input multi-output structure (i.e., FA) is correctly captured. Finally, BoolE converts the extracted e-graph into AIG format. And the output AIG can also be integrated into other tools (e.g., PolyCleaner and RevSCA [19], [20]) for further applications such as functional verification.

A. BoolE Methodology - Symbolic Reasoning via Equality Saturation

To identify FAs and reconstruct adder tree structures, BoolE employs equality saturation to process the input netlists. Here, we introduce the core components of the framework: e-graph construction and rewriting ruleset.

1) E-Graph Construction: BoolE gathers functional information from input netlists to accurately represent the AIG structure and construct the e-graph based on this information. The algorithm for e-graph construction is detailed in Algorithm 1. Firstly, BoolE collects data on the primary inputs and primary outputs of the netlists. Furthermore, BoolE establishes the dependency relationships for each Boolean operation, defining the input and output edges for the nodes. Subsequently, BoolE incrementally inserts operation nodes into the e-graph following the topological order of the graph nodes. Specifically, operations are inserted into the e-graph from the leaf nodes to the root nodes to ensure that child nodes are processed beforehand. This topological insertion order is critical for accurately maintaining the dependencies between operations. Each insertion generates an identifier representing the e-nodes and connects these e-nodes with their respective input parameters. The e-graph construction is shown in part ① in Figure 2.

Algorithm 1 E-graph Construction

Input: Vertex list V from parsing netlist
Output: : e-graph G_e
1: Initialize $vmap$ as empty HashMap
2: **for** each node v in TopoSort(V) **do** ▷ From leaf to root
3: $in\_id \leftarrow []$
4: **for** each input i for node v **do**
5: in_id.pushback($vmap[i]$) ▷ All children are inserted
6: **end for**
7: $id \leftarrow G_e$.insert(v, in_id) ▷ Insert v to e-graph
8: $vmap[v] = id$
9: **end for**

2) E-Graph Rewriting: The rulesets are composed of two parts: (1) basic Boolean rules (R_1), such as the commutative law, associative law, and De Morgan's Laws, which aim to expand the e-graph by saturating it with more functionally equivalent expression trees; (2) rules for `XOR` and `MAJ` identification (R_2), which directly identify `XOR` and `MAJ` operations within the e-graph. A subset of these rewriting rules is presented in Table I. In total, we collected 68 rules in R_1 and 119 rewriting rules for $R2$. To construct R_2, we utilized 8- to 128-bit Booth and CSA multipliers as templates and employed the synthesis tool ABC to detect the adder trees within the netlists and extract the structural patterns of `MAJ` and `XOR` operations. We constructed the corresponding e-graph rewriting rules R_2 and eliminated duplicate rules from the rule set.

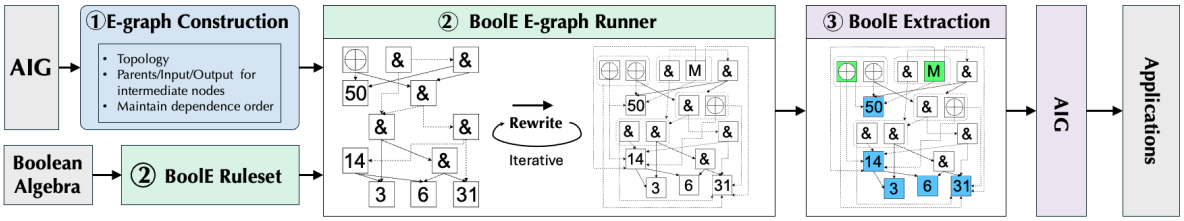


Fig. 2: The overview of BoolE, an exact symbolic reasoning framework for Boolean netlists utilizing equality saturation. It enhances performance by employing a novel extraction algorithm specifically designed to identify and capture multi-input, multi-output exact FA structure. We incorporate part of the e-graph for motivating example shown in Figure 1 to illustrate how the additional extra FA is extracted from e-graph, where the extracted e-nodes are specially highlighted. The XOR and MAJ gates forming an exact FA are highlighted in green.

The ruleset construction and e-graph rewriting correspond to part ② in Figure 2. Here we introduce 3 optimization tricks we employ in e-graph rewriting:

1) To enhance scalability while keeping performance, BoolE offers a lightweight version of rewriting rules. This version is carefully designed through empirical, manual pruning to ensure effectiveness for large-scale benchmarks, striking a balance between reducing complexity and maintaining reasoning performance.
2) Additionally, the two parts of the rewriting rules are applied incrementally in two phases. Firstly, we first apply R_1 to the initial e-graph with 10 iterations of rewriting. Subsequently, based on the R_1 rewrited e-graph, we apply R_2 with 3 iterations. This incremental saturation approach allows fine adjustment of the iteration limits for R_1 and R_2 separately.
3) After e-graph saturation, BoolE deletes redundant e-nodes that do not contribute to performance improvements. This reduces unnecessary memory usage and runtime. For instance, based on the commutative property, expressions such as $\texttt{XOR}(a, b, c)$, $\texttt{XOR}(b, a, c)$ belong to the same e-class. BoolE retains only one unique expression within each e-class, deleting the other equivalent e-nodes. A similar approach is applied to $\texttt{MAJ}(a, b, c)$ and $\text{FA}(a, b, c)$ expressions.

TABLE I: Example BoolE rewriting rules. The complete rewriting library consists of 68 Boolean rewriting rules and 39/90 additional rules specifically designed for the identification of MAJ/XOR operations.

	Pattern	Transformation
Basic rules	$a \,\&\, b$	$b \,\&\, a$
	a	$(\,a\,')\,'$
	$(a \,\&\, b)'$	$a' \,||\, b'$
	$(a \,||\, b)'$	$a' \,\&\, b'$
	$(a \,\&\, b) \,\&\, c$	$a \,\&\, (b \,\&\, c)$
MAJ rules	$(a \,\&\, b) \,||\, (a \,\&\, c) \,||\, (b \,\&\, c)$	$\texttt{MAJ}(a, b, c)$
	$(a' \,\&\, (b \,\&\, c)')' \,\&\, (b' \,\&\, c')'$	$\texttt{MAJ}(a, b, c)$
XOR rules	$(a\&b'\&c') \,||(a'\&b\&c') ||(a'\&b'\&c) ||(a\&b\&c)$	$\texttt{XOR}(a, b, c)$
	$(((a\|\|(b\&c)) \,||(b||c)') \,\&\, (a'|| \,((b\&c)'\&(b||c))))'$	$\texttt{XOR}(a, b, c)$

B. BoolE E-Graph Exact Extraction

In this section, we introduce our e-graph extraction for multi-input multi-output FA structure, which corresponds to part ③ in Figure 2.

Cost Function: Our cost function aims to maximize the number of exact FAs in the extracted expression within the e-graph G. Specifically, for an expression tree t_e composed of e-nodes e, the cost function $C(t_e)$ is defined as: $C(t_e) = \sum_{e \in t_e} -1_{\{\text{e is exact FA}\}}$. E-graph extraction seeks the optimal expression tree t^* with the lowest total cost from $\mathcal{T}$, the set of all possible expression trees in G:

$$t^* = \arg\min_{t_e \in \mathcal{T}} C(t_e)$$

Each expression tree t is constructed by selecting one node from each e-class, ensuring conformity to the e-graph structure.

Multi-output FA structure: Standard e-graph structures typically employ prefix notation expressions, assuming that all operators have single outputs. Consequently, the e-graph framework inherently lacks support for multi-output operations, only focusing on single-output functionalities. However, FA is a multi-input, multi-output structure, consisting of three inputs and two outputs representing the carry and sum, respectively.

To accommodate multi-output operations, equality saturation introduces new e-nodes into the e-graph. BoolE iterates through all e-nodes, identifying XOR and MAJ nodes with the exact same inputs. When such nodes are found, BoolE inserts an FA node that shares the same inputs and generates a tuple of outputs for carry and sum. Subsequently, BoolE employs the FST and SND pseudo-operations to project the individual carry and sum signals from the output tuple. Finally, BoolE unifies the e-classes of the FST and SND operations with the corresponding XOR and MAJ nodes, ensuring their equivalence within the e-graph framework. The structure of the FA is depicted in Figure 3. It is crucial to note that extracting FA structures is treated as an atomic operation. Thus, FST, SND, and FA must be selected collectively or not at all. This atomic extraction ensures the integrity of the FA structure is maintained during BoolE extraction.

DAG based extraction algorithm: BoolE utilizes DAG cost extraction, which counts each shared sub-expression only once for greater accuracy, to prevent double-counting of FA numbers. The extraction process is detailed in Algorithm 2. The algorithm first initializes a queue with all leaf nodes and an empty cost map for each e-class. It then iteratively processes nodes by dequeuing them and checking if all child nodes have computed costs. If so, it calculates the new cost for the node. If the new cost is lower than the previous cost, the cost map is updated, and parent nodes are enqueued. Nodes labeled FST/SND, and FA are handled specially in a post-processing step to ensure the integrity of FA structures. Each e-class maintains a cost map with the current best cost, selected e-node, and chosen child e-classes. By iteratively updating costs, the algorithm efficiently constructs the optimal expression tree t^*. This hash map, mapping child e-class IDs to their corresponding costs, introduces memory usage issues as the input e-graph grows. To save memory, BoolE uses flexible data types for the keys and values in the cost hash map. For small e-graphs (fewer than 65,536 e-classes), BoolE uses unsigned 16-bit integers for hash map keys, conserving memory. For larger e-graphs, it adjusts to unsigned 32-bit integers. This adaptability based on e-graph size ensures efficient memory utilization.

V. EXPERIMENTAL RESULTS

The experiments were conducted on a system with an Intel(R) Xeon(R) Gold 6418H CPU, featuring 48 physical cores and 2 threads per core, and 1 TByte of RAM. We use the *egg* framework [36]

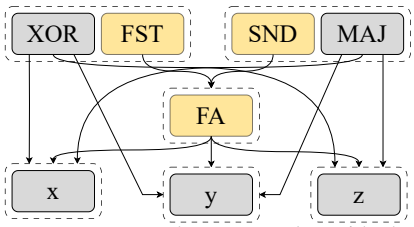


Fig. 3: FA Structure: XOR and MAJ e-nodes with the exactly same inputs will be paired. Each paired XOR/MAJ operation will insert a FA nodes and utilize *FST/SND* to project out the carry/sum bits. *FST/SND* will unified to the e-classes of XOR/MAJ

Algorithm 2 E-Graph Extraction

Input: e-graph G_e **Output:** Optimal t^* w.r.t. $Cost(G_e)$

```
function CalculateCostSet(G_e, node, Costs)
    results ← {key:Costs[key].results} for key in node.children()
    total ← Sum(result.values())
    return {results,total,node}
end function
function Extract(G_e, root)
    Queue ← LeafNode(G_e)                    ▷ Queue of e-nodes to be processed
    t* = ∅                                   ▷ Expression tree to be constructed
    Costset = ∅                              ▷ Initial empty map of e-class to cost
    while Queue is not empty do
        node = Queue.pop()
        if all c in node.children() are in Costset then
            prev_cost = Costset[class_id].total
            new_cost = CalculateCostSet(G_e, node, Costset)
            if new_cost.total < prev_cost then
                Queue.insert(node.parents())          ▷ Cost map update
                Costset[node]= new_cost
                if node is FST/SND then
                    PostProcessing()                  ▷ FA structure integrity check
                end if
            end if
        end if
    end while
end function
```

as the back-end to develop **BoolE** for e-graph construction and term rewriting. The experimental benchmarks include unsigned CSA multipliers and signed Booth multipliers, collected from the state-of-the-art (SOTA) baseline works ABC [42] and Gamora [37]. Specifically, the AIG-based approach (`&atree`) in ABC leverages structural hashing and functional pattern matching that reconstructs the adder tree structure via the cut enumeration algorithm [42]. Gamora is a SOTA graph learning-based Boolean reasoning tool [37] that targets the same objective, while the training dataset is collected with the AIG-based approach [42].

A. Performance of Symbolic Reasoning

To demonstrate the performance of BoolE, we prepared three research questions(RQs) in this section:

RQ1: How effective of BoolE without structure rewriting? – We collected the number of FAs identified by ABC and BoolE across various bitwidths for pre-mapping netlists. All adder tree structures are present in the netlists before technology mapping or logic optimization. For instance, in an n-bit CSA multiplier, the theoretical upper bound of FA number is $(n-1)^2-1$, which can be determined through exhaustive cut enumeration.

ABC can identify all adder trees in pre-mapping netlists successfully. Figure 4 presents the results alongside the theoretical upper bound for the number of pre-mapping FAs. In this figure, the upper bound curve data also represents the number of pre-mapping FAs identified by ABC and BoolE. **And BoolE also achieves the optimal solution for both CSA and Booth benchmarks across all bitwidths for pre-mapping benchmarks.** Since all NPN FAs can be exhaustively identified in pre-mapping netlists without rewriting, the reasoning performance is entirely dominated by ruleset R_2 described in Section IV-A2, which is specialized for XOR and MAJ identification. The result underscores the superior performance of ruleset R_2, which dominates the reasoning capability to discover all existing NPN FAs in pre-mapping netlists.

RQ2: How effective is BoolE under heavily logic optimization and technology mapping? – Reasoning high-level components under heavily logic optimization and technology mapping settings is known to be significant more challenging [37] [42] [12] [18], which is more critical in real-world datapath synthesis [10], [41] and formal verification scenarios [21], [40], [42]. Therefore, our objective is to demonstrate the robustness of BoolE under technology mapping and logic optimization, which reflects the rewriting and extraction capability of BoolE.

To evaluate the performance of BoolE on technology-mapped netlists, we quantified the number of FAs discovered by BoolE across technology-mapped CSA and Booth multipliers of various bitwidths. We utilized the synthesis tool ABC to perform technology mapping with the 7nm ASAP library, which encompasses a diverse set of 161 standard-cell gates. The test results are presented in Figure 4, with the CSA and Booth multiplier results shown in the left and right subfigures, respectively. The x-axis represents the bitwidth of the benchmark multipliers, while the y-axis indicates the number of FAs discovered using different reasoning tools: BoolE, ABC, and Gamora.

The results demonstrate that BoolE successfully reconstructs the majority of FA structures, and provides superior exact symbolic reasoning performance for multipliers compared to ABC and Gamora. Later in Section V-B, we also demonstrate its robustness against formal verification tool RevSCA-2.0, which integrates the same reasoning function. **Specifically, BoolE achieves an average of** 93.48% **and** 84.81% **of the theoretical upper bound for NPN FAs in technology-mapped CSA and Booth multipliers, respectively.** In comparison, ABC can only identify 68.07% and 69.28% of the maximum NPN FAs for CSA and Booth multipliers on average, respectively. Furthermore, Gamora reasoning performance drops to 63.81% and 66.68% for CSA and Booth multipliers, respectively. BoolE constantly outperforms ABC and Gamora for post-mapping netlists. Additionally, BoolE focuses on reconstructing as many exact FAs as possible. **The number of exact FAs after BoolE rewriting increased by** $3.53\times$ **and** $3.01\times$ **compared with ABC for CSA and Booth multipliers, respectively.**

RQ3: Is BoolE scalable? – To demonstrate the scalability of BoolE, we present the end-to-end runtime across all tested benchmarks. Specifically, we recorded the runtime and number of nodes in the netlist of each benchmark, including post-mapping CSA and Booth multipliers. The results are illustrated in Figure 5.

Although functional rewriting for Boolean networks involves higher time complexity compared to simple structural matching in cut enumeration, BoolE achieves efficient rewriting runtimes. It completes all benchmarks for post-mapping Booth and CSA multipliers from 4-bit to 128-bit within 50 minutes. For the largest case, 128-bit post-mapping CSA multipliers containing 191,863 AIG nodes, the rewriting process takes 50 minutes. The runtime for BoolE rewriting is insignificant compared to the order-of-magnitude improvements it brings to real-world applications. In tasks such as formal verification, the reasoning performance directly impacts the overall complexity and runtime. BoolE's superior reasoning performance enables orders of magnitude speedup in the formal verification of larger, optimized multipliers. For example, BoolE takes only 11 seconds to reason the netlist for a 24-bit multiplier, while the verification runtime is drastically reduced from 32,402.74 seconds to just 0.07 seconds. The runtime spent on the

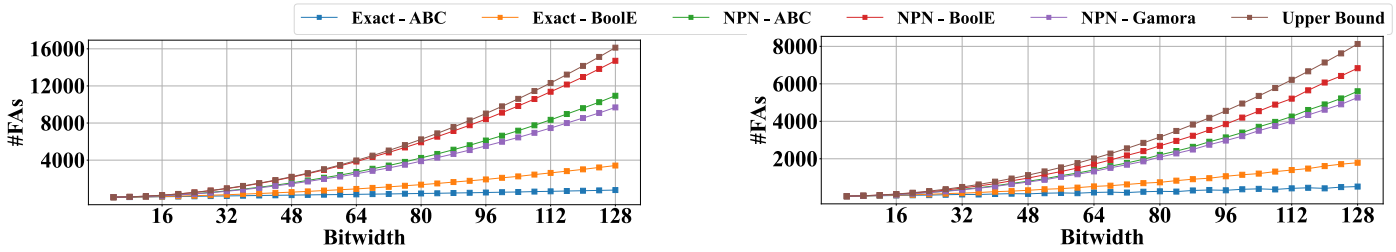


Fig. 4: Performance comparison among BoolE, ABC, and Gamora for CSA (left) and Booth (right) multipliers after ASAP 7nm technology mapping. BoolE consistently outperforms ABC and Gamora, identifying 3.53× and 3.01× exact FAs than ABC in CSA and Booth multipliers. And BoolE achieves an average of 93.48% and 84.81% of the theoretical upper bound in number of NPN FAs, respectively.

rewriting process is well compensated by the substantial reductions achieved in end-to-end verification time. The runtime and efficiency for formal verification tasks will be elaborated in Section V-B.

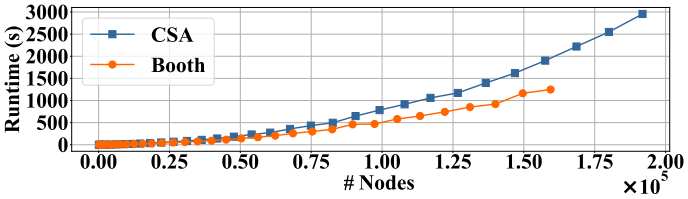


Fig. 5: BoolE runtime of post-mapping CSA and Booth multiplier w.r.t. the size of the input netlist (AIG node number).

B. Integrated Application to Formal Verification

RQ4: Can BoolE practically enhance real-world applications orthogonally (formal verification)? Boolean symbolic reasoning is a critical topic in various domains, such as functional verification and datapath optimization. We take verification of multipliers as a case study to illustrate how BoolE can enhance real-world applications. Boolean symbolic reasoning, as an orthogonal technique, can be integrated into many formal verification tools for reverse engineering [20], [21], [28]. Here we utilize RevSCA-2.0 [21] as our back-end tool. The complexity of symbolic computer algebra-based multiplier verification is determined by the maximum polynomial size (# monomials) [40], and its performance is measured by verification runtime. To mitigate polynomial explosion, RevSCA-2.0 uses cut enumeration to detect functional blocks, including exact FAs. It then eliminates vanishing monomials within these blocks before backward rewriting. Consequently, reasoning performance directly impacts the detection of functional blocks, the elimination of vanishing monomials, and the overall success of verification.

ABC offers a logic circuit optimization feature that simplifies Boolean expressions and reduces gate counts. This optimization functionality is integrated into the *dch* interface. We evaluated the total runtime of verifying a CSA multiplier benchmark, optimized using *dch* bit-optimization, under two configurations: with and without BoolE optimization. Note that RevSCA-2.0 is equipped with a functional reasoning engine with cut enumeration. The test results are presented in Table II. This table provides the verification runtime across various bitwidths. Additionally, we report the maximum polynomial size (Max Poly Size) observed during the backward iteration process, which reflects the complexity of the verification tasks. It is important to note that we only report the number of exact FAs because NPN FAs cannot be detected as functional blocks, and only exact FAs are beneficial for multiplier verification simplification. Verification runtime exceeding 72 hours is marked as time out (TO).

We can see a significant improvement in runtime brought by BoolE rewriting. The formal verification of CSA multipliers after logic optimization gets timed-out without BoolE rewriting when the bitwidth is larger than 24 bits. After BoolE rewriting, the multipliers of up to 128 bit can be formally verified. And there is a significant speedup in verifying multipliers with higher bitwidths. For example, BoolE provides 2825× runtime improvement for 24-bit *dch*-optimized multiplier via exact symbolic reasoning. Furthermore, BoolE enables the verification of *dch*-optimized multipliers with bitwidths exceeding 24 bits. However, without BoolE, RevSCA-2.0 cannot formally verify these larger multipliers within a **72-hour timeout**.

TABLE II: Verification results of `dch`-optimized CSA multipliers using RevSCA-2.0 under two configurations: with BoolE optimization (BoolE) and without optimization (Baseline).

Bitwidth	Number of Exact FAs			Max Poly Size		End-to End Runtime (s)	
	Upper Bound	BoolE	Baseline	BoolE	Baseline	BoolE	Baseline
8	48	41	0	85	373	0.445	0.035
12	120	109	0	173	2173	0.955	0.297
16	224	209	0	293	34345	2.474	17.332
20	360	341	0	445	548917	5.165	1471.574
24	528	505	0	629	8781889	11.470	32402.739
28	728	701	0	845	-	22.951	TO
32	960	929	0	1093	-	43.910	TO
64	3968	3905	0	4229	-	1232.219	TO
96	9024	8929	0	9413	-	8748.443	TO
128	16128	16001	0	16645	-	33255.292	TO

As illustrated in the fourth column of Table II, the number of exact FAs detected by ABC in CSA multipliers drops to zero due to logic optimization. This disappearance of exact FAs significantly increases the complexity of the verification process during backward rewriting, especially as the bitwidth scales. On the other hand, BoolE can reconstruct most disappeared functional blocks. In the third column of Table II, we present the number of exact FAs reconstructed by BoolE, which is compared with the upper bound in the second column which represents the maximum number of FAs in CSA multipliers. BoolE successfully reconstructs up to 99.2% of the exact FAs within the theoretical upper bound.

VI. Conclusion

In conclusion, this paper introduces BoolE, an exact Boolean symbolic reasoning framework that leverages equality saturation techniques in e-graphs to overcome the limitations of conventional methods. By assembling a comprehensive ruleset and implementing careful optimizations, BoolE achieves enhanced performance in Boolean symbolic reasoning, enabling accurate Boolean function detection. BoolE effectively reconstructs multi-output structures, such as FAs, within technology-mapped and optimized netlists. Our evaluations demonstrate BoolE's significant improvements, highlighting its potential to advance digital circuit synthesis and verification.

Acknowledgment – This work is supported by National Science Foundation (NSF) under CCF2350186, CCF2403134, CCF2349670, and CCF2349461 awards.

REFERENCES

[1] L. Alrahis, A. Sengupta, J. Knechtel, S. Patnaik, H. Saleh, B. Mohammad, M. Al-Qutayri, and O. Sinanoglu, "Gnn-re: Graph neural networks for reverse engineering of gate-level netlists," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 41, no. 8, pp. 2435–2448, 2021.

[2] A. Biere, "Lingeling, Plingeling and Treengeling Entering the SAT Competition 2013," *SAT Competition*, 2013.

[3] R. Brayton and A. Mishchenko, "Abc: An academic industrial-strength verification tool," in *Computer Aided Verification: 22nd International Conference, CAV 2010, Edinburgh, UK, July 15-19, 2010. Proceedings 22*. Springer, 2010, pp. 24–40.

[4] Y. Cai, K. Yang, C. Deng, C. Yu, and Z. Zhang, "Smoothe: Differentiable e-graph extraction," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, 2025, pp. 1020–1034.

[5] B. Cakir and S. Malik, "Reverse engineering digital ics through geometric embedding of circuit graphs," *ACM Transactions on Design Automation of Electronic Systems (TODAES)*, vol. 23, no. 4, pp. 1–19, 2018.

[6] C. Chen, G. Hu, D. Zuo, C. Yu, Y. Ma, and H. Zhang, "E-syn: E-graph rewriting with technology-aware cost functions for logic synthesis," in *ACM/IEEE Design Automation Conference*, 2024, pp. 1–6.

[7] M. Ciesielski, T. Su, A. Yasin, and C. Yu, "Understanding algebraic rewriting for arithmetic circuit verification: a bit-flow model," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 39, no. 6, pp. 1346–1357, 2019.

[8] M. Ciesielski, C. Yu, W. Brown, D. Liu, and A. Rossi, "Verification of gate-level arithmetic circuits by function extraction," in *Proceedings of the 52nd Annual Design Automation Conference*, 2015, pp. 1–6.

[9] S. Coward, G. A. Constantinides, and T. Drane, "Automatic datapath optimization using e-graphs," in *2022 IEEE ARITH*. IEEE, 2022, pp. 43–50.

[10] S. Coward, T. Drane, E. Morini, and G. A. Constantinides, "Combining power and arithmetic optimization via datapath rewriting," in *2024 IEEE 31st Symposium on Computer Arithmetic (ARITH)*. IEEE, 2024, pp. 24–31.

[11] L. De Moura and N. Bjørner, "Satisfiability Modulo Theories: Introduction and Applications," *Communications of the ACM*, 2011.

[12] C. Deng, Z. Yue, C. Yu, G. Sarar, R. Carey, R. Jain, and Z. Zhang, "Less is more: Hop-wise graph attention for scalable and generalizable learning on circuits," in *Proceedings of the 61st ACM/IEEE Design Automation Conference*, 2024, pp. 1–6.

[13] Z. He, Z. Wang, C. Bai, H. Yang, and B. Yu, "Graph learning-based arithmetic block identification," in *2021 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*. IEEE, 2021, pp. 1–8.

[14] Z. Huang, L. Wang, Y. Nasikovskiy, and A. Mishchenko, "Fast boolean matching based on npn classification," in *2013 International Conference on Field-Programmable Technology (FPT)*. IEEE, 2013, pp. 310–313.

[15] D. Kaufmann and A. Biere, "Amulet 2.0 for verifying multiplier circuits," in *International Conference on Tools and Algorithms for the Construction and Analysis of Systems*. Springer, 2021, pp. 357–364.

[16] B. W. Leverett, R. G. G. Cattell, S. O. Hobbs, J. M. Newcomer, and et. al, "An overview of the production quality compiler-compiler project," *Computer*, vol. 13, no. 8, pp. 38–49, 1980.

[17] M. Li, S. Khan, Z. Shi, N. Wang, H. Yu, and Q. Xu, "Deepgate: Learning neural representations of logic gates," in *Proceedings of the 59th ACM/IEEE Design Automation Conference*, 2022, pp. 667–672.

[18] W. Li, A. Gascon, P. Subramanyan, W. Y. Tan, A. Tiwari, S. Malik, N. Shankar, and S. A. Seshia, "Wordrev: Finding word-level structures in a sea of bit-level gates," in *HOST*. IEEE, 2013, pp. 67–74.

[19] A. Mahzoon, D. Große, and R. Drechsler, "Polycleaner: clean your polynomials before backward rewriting to verify million-gate multipliers," in *2018 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. IEEE, 2018, pp. 1–8.

[20] ——, "Revsca: Using reverse engineering to bring light into backward rewriting for big and dirty multipliers," in *Proceedings of the 56th Annual Design Automation Conference 2019*, 2019, pp. 1–6.

[21] ——, "Revsca-2.0: Sca-based formal verification of nontrivial multipliers using reverse engineering and local vanishing removal," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 41, no. 5, pp. 1573–1586, 2021.

[22] T. Meade, S. Zhang, Y. Jin, Z. Zhao, and D. Pan, "Gate-level netlist reverse engineering tool set for functionality recovery and malicious logic detection," in *International Symposium for Testing and Failure Analysis*, vol. 81368. ASM International, 2016, pp. 342–346.

[23] G. Nelson and D. C. Oppen, "Fast decision procedures based on congruence closure," *Journal of the ACM (JACM)*, vol. 27, no. 2, pp. 356–364, 1980.

[24] R. Nieuwenhuis and A. Oliveras, "Proof-producing congruence closure," in *International Conference on Rewriting Techniques and Applications*. Springer, 2005, pp. 453–468.

[25] P. Pan and C.-C. Lin, "A new retiming-based technology mapping algorithm for lut-based fpgas," in *Proceedings of the 1998 ACM/SIGDA sixth international symposium on Field programmable gate arrays*, 1998, pp. 35–42.

[26] S. Rajendran and M. L. Regeena, "A novel algorithm for hardware trojan detection through reverse engineering," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 41, no. 4, pp. 1154–1166, 2021.

[27] V. Rao and Z. D. Sisco, "Register aggregation for hardware decompilation," *arXiv preprint arXiv:2409.03119*, 2024.

[28] D. Ritirc, A. Biere, and M. Kauers, "Improving and extending the algebraic approach for verifying gate-level multipliers," in *2018 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 2018, pp. 1556–1561.

[29] Z. Shi, Z. Zheng, S. Khan, J. Zhong, M. Li, and Q. Xu, "Deepgate3: towards scalable circuit representation learning," *arXiv preprint arXiv:2407.11095*, 2024.

[30] Z. D. Sisco, J. Balkind, T. Sherwood, and B. Hardekopf, "Loop rerolling for hardware decompilation," *Proceedings of the ACM on Programming Languages*, vol. 7, no. PLDI, pp. 420–442, 2023.

[31] P. Subramanyan, N. Tsiskaridze, K. Pasricha, D. Reisman, A. Susnea, and S. Malik, "Reverse engineering digital circuits using functional analysis," in *2013 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 2013, pp. 1277–1280.

[32] R. Tate, M. Stepp, Z. Tatlock, and S. Lerner, "Equality saturation: a new approach to optimization," in *Proceedings of the 36th annual ACM SIGPLAN-SIGACT symposium on Principles of programming languages*, 2009, pp. 264–276.

[33] E. Ustun, I. San, J. Yin, C. Yu, and Z. Zhang, "Impress: Large integer multiplication expression rewriting for fpga hls," in *2022 IEEE 30th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*. IEEE, 2022, pp. 1–10.

[34] E. Ustun, C. Yu, and Z. Zhang, "Equality saturation for datapath synthesis: A pathway to pareto optimality," in *2023 60th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2023, pp. 1–2.

[35] Z. Wang, Z. He, C. Bai, H. Yang, and B. Yu, "Efficient arithmetic block identification with graph learning and network-flow," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 42, no. 8, pp. 2591–2603, 2022.

[36] M. Willsey, C. Nandi, Y. R. Wang, O. Flatt, Z. Tatlock, and P. Panchekha, "Egg: Fast and extensible equality saturation," *Proceedings of the ACM on Programming Languages*, vol. 5, no. POPL, pp. 1–29, 2021.

[37] N. Wu, Y. Li, C. Hao, S. Dai, C. Yu, and Y. Xie, "Gamora: Graph learning based symbolic reasoning for large-scale boolean networks," in *2023 60th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2023, pp. 1–6.

[38] X. Xu, N. Shah, A. Evans, S. Sinha, B. Cline, and G. Yeric, "Standard cell library design and optimization methodology for asap7 pdk," in *2017 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. IEEE, 2017, pp. 999–1004.

[39] Y. Yang, P. Phothilimthana, Y. Wang, M. Willsey, S. Roy, and J. Pienaar, "Equality saturation for tensor graph superoptimization," *Proceedings of Machine Learning and Systems*, vol. 3, pp. 255–268, 2021.

[40] C. Yu, W. Brown, D. Liu, A. Rossi, and M. Ciesielski, "Formal verification of arithmetic circuits by function extraction," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 35, no. 12, pp. 2131–2142, 2016.

[41] C. Yu and M. Ciesielski, "Automatic word-level abstraction of datapath," in *2016 IEEE International Symposium on Circuits and Systems (ISCAS)*. IEEE, 2016, pp. 1718–1721.

[42] C. Yu, M. Ciesielski, and A. Mishchenko, "Fast algebraic rewriting based on and-inverter graphs," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 37, no. 9, pp. 1907–1911, 2017.

[43] Y. Zhang, Y. R. Wang, O. Flatt, D. Cao, P. Zucker, E. Rosenthal, Z. Tatlock, and M. Willsey, "Better together: Unifying datalog and equality saturation," *Proceedings of the ACM on Programming Languages*, vol. 7, no. PLDI, pp. 468–492, 2023.